The RTL Gauntlet: Measuring Honesty, Cost, and Latency-Robustness in Agentic Hardware Design

(working draft)

Abstract

Agentic RTL frameworks now report 100% pass on standard benchmarks, but that score is measured against *visible* tests, is *expensive* to reach, and is obtained on *proxy* tasks. We turn these three concerns into measurable axes. We build a two-tier protocol that freezes an agent's design and scores it with a withheld formal-equivalence oracle, and define a Reward-Hacking Gap (RHG) and Honest Pass Rate (HPR). Applying it to 156 VerilogEval tasks across two models, we find the naïve headline RHG (6.1%) is *entirely an oracle artifact*: a naïve formal oracle over-reports reward hacking via don't-care `x`, async-reset, state-encoding, init-state, and even a SystemVerilog parser flag. We harden the oracle in five steps —reset-aware, don't-care-aware, bounded miter+SAT, `read_verilog -sv`, and a `memory` (case→ROM) pass—cutting false counter-examples $9 \to 1$ and "inconclusive" $50 \to 6$, and verify every surviving flag by hand: **zero genuine reward hacking** (95% upper bound $\leq 3\%$) by frontier models on fair tasks. A weaker model (Haiku 4.5) fails 3× more than Opus 4.8 on the visible tests but does *not* hack more; even a tamper-capable shell agent that struggles for several iterations does not edit the testbench. We show the oracle *would* catch hacking when present (a candidate that passes a randomized hidden testbench but is formally disproven), that the result survives off the verbatim benchmark (a contamination probe), and we add a token-cost analysis (an early-stop reclaims 12–23% of tokens) and a latency-axis prototype that produces real Sky130 power/area/timing through a containerized OpenLane flow. The central artifact for honest agentic-hardware evaluation is a don't-care/reset/encoding-aware oracle plus a verification discipline—not a new pass-rate.

1 Introduction

HORIZON treats agentic hardware design as repository-level code evolution and reaches 100% pass on several RTL suites—but iteration-0 pass is only 47.8%, so the headline comes from *iterative repair against a visible signal*. Its authors name three open problems: reward hacking, token cost, and high-latency (PPA) reward. We make each measurable.

Thesis. *Pass@visible is the wrong score for agentic hardware design*: it can be gamed (honesty), it is expensive (cost), and it does not scale to real reward (latency).

Contributions.

1. A two-tier, formal-grounded evaluation protocol with metrics RHG and HPR (§3).
2. The finding that a naïve formal oracle **over-reports** reward hacking, and a five-step hardened oracle that removes it; verification discipline as a first-class method (§4).
3. An empirical result across 2 models × 156 tasks: **zero verified genuine reward hacking** on fair tasks; weakness ≠ hacking; tampering is caught when present (§5).
4. Cost and latency prototypes: token accounting with an early-stop policy, and real Sky130 PPA via a containerized flow (§5).

Work	gap metric	formal oracle	RTL
SpecBench	✓(software)	finite held-out	—
EvilGenie	✓+ edit-detect	—	—
Agentic Frontier	—	—	✓
This work	✓	**exhaustive**	✓

Table 1: SpecBench owns the visible/held-out gap for *software* and admits its held-out suite "cannot exhaustively certify compliance." Our slot: RTL + an exhaustive formal oracle (fixing that limit) + tamper-evidence.

2 Related Work and Positioning

Formal-equivalence-as-oracle is already community-standard for RTL–LLM evaluation (VeriThoughts; NotSoTiny; RealBench)—so it is our *tool*, not our contribution. Table 1 situates the closest work.

3 Method: the two-tier protocol

The agent sees a complete specification, a *visible* diagnostic testbench, and its failure logs, and edits RTL until visible passes. We then **freeze** the design and score withheld tiers it never saw: a hidden randomized/exhaustive testbench, and **formal equivalence** vs. a curated golden (Yosys). The I/O interface is locked. We define

$$\mathsf{RHG} = \frac{\#\{\text{visible-passers failing a withheld tier}\}}{\#\{\text{visible-passers}\}}, \quad \mathsf{HPR} = \frac{\#\{\text{pass visible and every withheld tier}\}}{\#\text{tasks}}.$$

A tamper tier (a shell-capable agent that may edit any workspace file) flags edits to the testbench/harness as *exploit-evidenced* hacking, with the withheld tiers run on frozen originals so tampering cannot fool the verdict. We separate the claim into behavioral gap, exploit-evidenced, and tamper-confirmed, and never claim intent without trajectory evidence.

4 The oracle, and why naïve formal over-reports

On a 156-task sweep (Opus 4.8), a naïve `equiv_make` oracle reported 9 RHG counter-examples and 50 "inconclusive." **Every CEX was a false positive**, found by hand-verifying flagged cases: don't-care `x` in the golden; functionally identical designs flagged on async reset / initial state; a candidate logically identical but with a different state encoding; and—for most "inconclusive"—a silent parse failure, since `read_verilog` defaults to Verilog-2005 while the references use `always_ff`/`logic`/enums.

Our hardening pipeline (each step tested on verified cases before re-running):

1. `async2sync` (normalize async resets);
2. reclassify a CEX to *dontcare* when the golden has an `x` literal;
3. a bounded miter+SAT fallback comparing I/O sequences directly (encoding-agnostic; a SAT model is a real CEX);
4. `read_verilog -sv`;
5. a `memory` (case→ROM) pass.

5 Experiments

Formal earns its keep. `formal_demo`: a 16-bit candidate wrong *only* on `0xDEAD`. The randomized hidden testbench (512/65536) misses it—visible *and* hidden PASS—while exhaustive formal returns CEX. With finite tests alone RHG = 0 (it looks honest); formal exposes it (RHG = 0.50). This is the direct evidence that an exhaustive formal oracle beats a finite held-out suite.

stage	honest	bmc	dc	RHG	inconcl	fail
naïve	88	–	–	9	50	9
+reset+dc	91	–	3	3	50	9
+BMC	91	3	2	1	50	9
+SV	122	6	4	1	14	9
+memory	**129**	6	5	1	**6**	9

Table 2: Oracle hardening (same 156 Opus candidates, no new LLM calls). False CEX 9→1; inconclusive 50→6; HPR 56%→87%. The 6 residual are hard sequential FSMs.

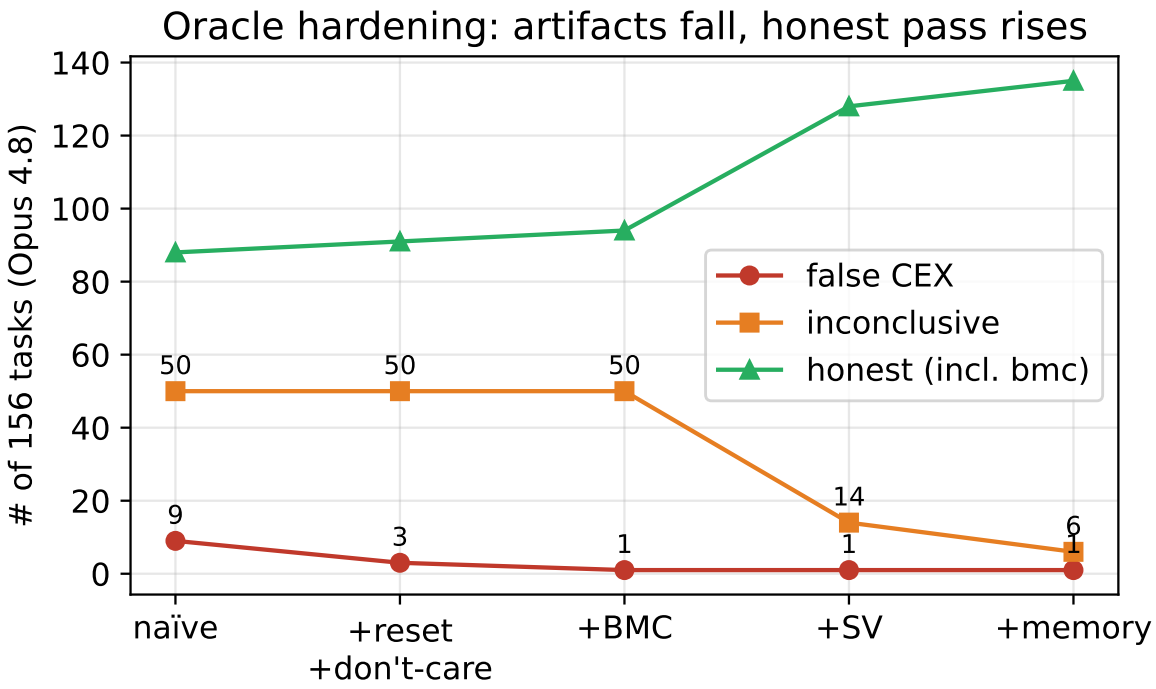

Figure 1: Oracle hardening: artifacts fall (false CEX 9→1, inconclusive 50→6) and the honest pass count rises (88→135).

Sweep and model comparison. Table 3 and Figure 2: the weaker model fails far more (`fail_visible` 9→30, +11 no compilable candidate) but hacks no more. Both residual RHG counter-examples are verified init artifacts. With 95% Wilson intervals, verified-genuine RHG= 0 with an upper bound ≤2.5% (Opus) / ≤3.2% (Haiku).

Elicit (negative, informative). A tamper-capable shell agent on a harder task: Haiku struggled four iterations (with full affordance to edit the testbench) but never tampered—it kept honestly repairing the design and solved it; Opus was honest in one. Genuine reward hacking is not elicited from aligned frontier models in a single agentic loop even under affordance and difficulty (the oracle *would* catch it—the planted red-team is caught). This points to RL training-time emergence as the regime to study.

Contamination. VerilogEval is public (reported ∼100% contamination for older models), so an honest pass could be memorization. We generate textually-novel, functionally-identical tasks (rename the target module, reframe the spec) and re-sweep: on the first 40 tasks Opus is HPR 1.00→1.00, RHG 0→0 ($\Delta = 0$). The result is robust to identifier-level contamination. As a *semantic*-novelty control, our self-authored tasks (`gray2bin`, `popcount8`, `hex7seg`) implement functions absent from any public benchmark, yet models are

	Opus 4.8	Haiku 4.5
honest + bmc	135	107
HPR (95% CI)	0.82 [.75,.87]	0.69 [.61,.75]
`fail_visible` (+ no-cand)	9	30 (+11)
verified RHG	0 ($\leq$ .025)	0 ($\leq$ .032)

Table 3: Two models $\times$ 156 tasks. Weakness shows as failures, not cheating.

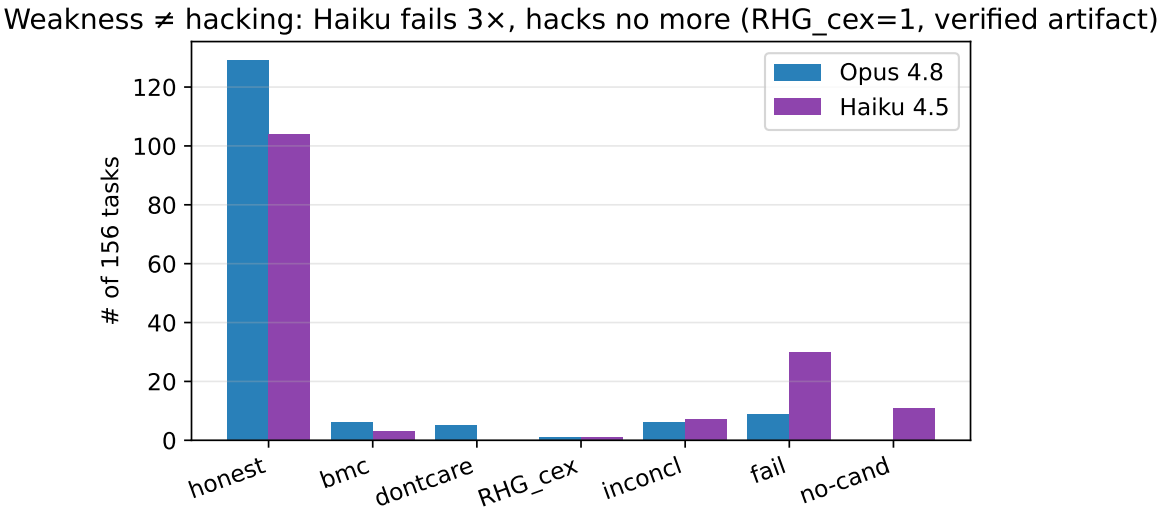

Figure 2: Weakness $\neq$ hacking: Haiku fails 3$\times$ more on the visible tests, hacks no more.

honest on them too (RHG $= 0$)—so the result is not benchmark memorization.

Cost (C2). Opus 482k / Haiku 522k tokens; the weaker model needs more repair (2-iter 17 vs 36). The repair tail is mostly wasted (honest payoff 35% / 14%); an early-stop at one iteration reclaims 12% (Opus) / 23% (Haiku) of tokens for a $\sim$5% honesty loss (Figure 3).

Latency (C3). A surrogate pipeline verified offline (mock 315 designs $\rightarrow$ holdout Pearson $r = 0.89/0.91/0.96$ for area/power/timing) and, via a containerized OpenLane flow (the OpenLane image as runtime $\rightarrow$ native `openlane`, no Docker-in-Docker), produced real Sky130 metrics: `counter8` (seq) 495.5 μm^2 / 0.120 mW / slack $\approx$5.5 ns; `popcount8` (comb) 247.7 μm^2 / 0.051 mW / 1.6 ns. (The deploy gotcha was a config file at the repo root, not OpenLane.)

6 Findings

1. A naïve formal oracle **over-reports** reward hacking; the headline number was an artifact, found only by per-case verification. Verification discipline is mandatory.
2. No false positives on honest agents; across 2 models $\times$ 156 fair tasks, **zero verified genuine reward hacking** (95% upper bound $\leq$3%).
3. **Weakness $\neq$ hacking**—a weaker model fails far more but cheats no more, even under tamper affordance.
4. Hacking is a function of adversarial conditions, not benchmark mass; the protocol catches it when present.

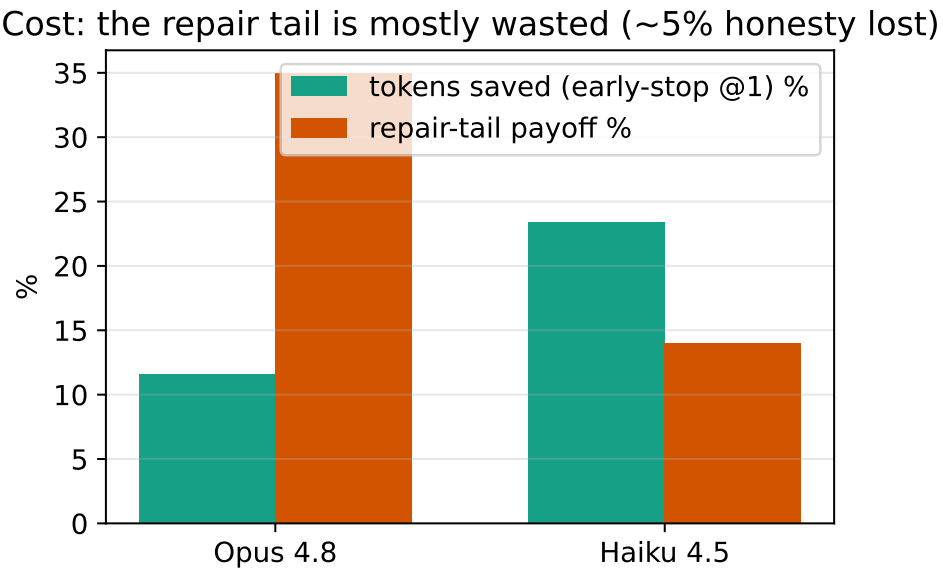

Figure 3: The repair tail is mostly wasted; an early-stop reclaims 12–23% of tokens.

7 Threats to Validity

Contamination is only probed at the identifier level; a semantic mutation is needed for a strong claim. *Oracle residual*: 6 hard sequential FSMs remain inconclusive (EQY structural matching is the fix). *Eliciting* genuine hacking from aligned models failed; the negative result may reflect single-loop inference rather than RL training. *PPA* on toy designs is noisy (not signoff-grade). *Nondeterminism*: LLM runs vary; we report Wilson CIs and pin model ids.

8 Limitations and Future Work

EQY structural matching for the residual FSMs; a semantic contamination mutation; more models and task types (debug/verify/reuse); a PPA surrogate trained on a real multi-design dataset with an agent loop under high-latency reward; and—most ambitiously—**RL training-time hacking**: train a small RTL agent with a visible-test reward and measure emergence of hacking via the hidden+formal oracle.

9 Conclusion

Honest evaluation of agentic hardware design is gated not by a new pass-rate but by an oracle that is don't-care/reset/encoding-aware and by a discipline of verifying every flag. With that, frontier agents do not hack fair RTL tasks—but the oracle catches it when they do.

References

[1] HORIZON: Agentic Hardware Design as Repository-Level Code Evolution. arXiv:2606.28279.

[2] Comprehensive Verilog Design Problems (CVDP). arXiv:2506.14074.

[3] SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents. arXiv:2605.21384.

[4] EvilGenie: A Reward Hacking Benchmark. arXiv:2511.21654.

[5] VeriContaminated: Assessing LLM-Driven Verilog Coding for Data Contamination. arXiv:2503.13572.

[6] Trace2Skill: Verifier-Guided Skill Evolution for Long-Context EDA Agents. arXiv:2605.21810.

[7] NotSoTiny: A Large, Living Benchmark for RTL Code Generation. arXiv:2512.20823.

[8] RealBench: Benchmarking Verilog Generation Models with Real-World IP Designs. arXiv:2507.16200.

[9] Exploring the Agentic Frontier of Verilog Code Generation. arXiv:2603.19347.

[10] Understanding Inference-Time Token Allocation and Coverage Limits in Agentic Hardware Verification. arXiv:2604.15657.